

Deploy Magento 2 bằng Jenkins - Phần 2

Jenkins chung VPS + GitHub Webhook that

Thang 9, 2026

Deploy Magento 2 bằng Jenkins — Phần 2: Jenkins chung VPS Magento + GitHub Webhook thật

Bối cảnh: Sau giai đoạn học với Jenkins chạy trên máy local (Phần 1), giờ chuyển Jenkins sang chạy trực tiếp trên VPS đang chạy Magento 2 (Ubuntu 22.04, nginx/PHP-FPM/OpenSearch cài native). Tạo subdomain riêng cho Jenkins, kết nối GitHub Webhook để build/deploy tự động ngay khi push code — không cần chờ Poll SCM.

1. Quy ước placeholder

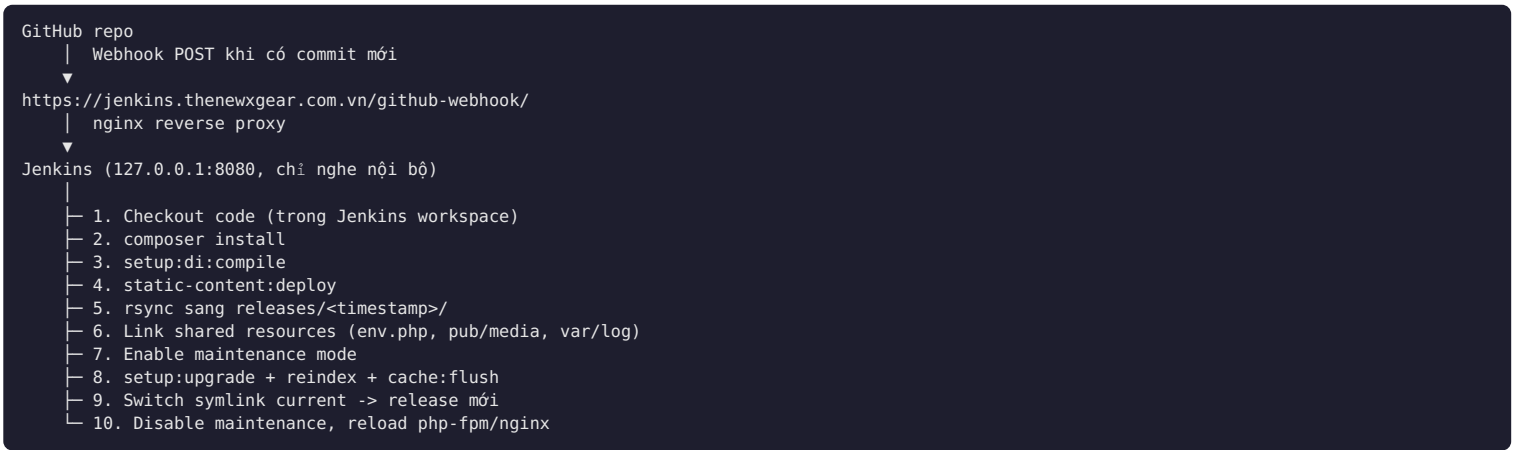
Placeholder	Ý nghĩa	Giá trị thực tế của bạn
<VPS_APP_IP>	IP VPS chạy Magento + Jenkins	IP VPS thật
<DOMAIN>	Domain chính	thenewxgear.com.vn
<JENKINS_SUBDOMAIN>	Subdomain dành cho Jenkins	jenkins.thenewxgear.com.vn
<PHP_FPM_SERVICE>	Tên service PHP-FPM	ví dụ php8.1-fpm
<GIT_REPO_URL>	URL repository Git	https://github.com/<user>/<repo>.git

2. Những gì thay đổi so với Phần 1 (Jenkins local)

Phần 1 (local + Poll SCM)	Phần 2 (chung VPS + Webhook)
Jenkins ở máy local	Jenkins cài ngay trên VPS Magento
Build local → scp / ssh sang VPS	Build và deploy chạy lệnh nội bộ, không cần SSH
Cần SSH key riêng (sshagent)	Không cần SSH key — bỏ hẳn sshagent trong Jenkinsfile
Trigger: Poll SCM mỗi vài phút	Trigger: GitHub gọi webhook — build gần như tức thời
Không cần public port	Cần expose Jenkins qua subdomain + SSL an toàn

Cấu trúc releases/shared/current trên VPS giữ nguyên 100% — không đổi gì ở phần Magento, chỉ đổi cách Jenkins tương tác với filesystem (nội bộ thay vì qua SSH).

3. Kiến trúc tổng thể



Lưu ý về thuật ngữ: pipeline này là **minimal-downtime deployment**, không phải zero-downtime tuyệt đối. Vẫn cần một khoảng

bảo trì ngắn (vài giây đến vài chục giây) trong lúc chạy `setup:upgrade`, vì code mới và schema database phải đồng bộ cùng lúc.

4. Cài Jenkins trên VPS Magento

4.1. Cài Jenkins

```
sudo apt update
sudo apt install -y openjdk-17-jre

curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/" | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update && sudo apt install -y jenkins
sudo systemctl enable --now jenkins
sudo systemctl status jenkins
```

Jenkins mặc định chạy port 8080, chỉ truy cập tạm qua SSH tunnel trong lúc cài đặt ban đầu (sẽ khóa hẳn port này ở Bước 8):

```
# Chạy trên máy cá nhân
ssh -L 8080:localhost:8080 root@VPS_APP_IP>
```

Mở `http://localhost:8080` trên máy cá nhân, lấy password khởi tạo:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Cài **Suggested plugins** → tạo admin user.

4.2. Cài thêm plugin cần thiết

Manage Jenkins → **Plugins** → **Available plugins**, tick chọn: - **Git** - **Pipeline** - **Credentials Binding** - **GitHub** (plugin tên chính xác “GitHub Plugin” — cung cấp endpoint `/github-webhook/` và trigger “GitHub hook trigger for GITScm polling”)

Không cần **SSH Agent** plugin — vì Jenkins và Magento chung 1 máy, mọi lệnh chạy trực tiếp (`sh`), không cần `sshagent` / `ssh` / `scp`.

Restart Jenkins sau khi cài xong plugin nếu được yêu cầu.

5. Cấu hình quyền cho user `jenkins`

Jenkins chạy dưới user hệ thống tên `jenkins` (tự tạo khi cài qua apt) — cần quyền đọc/ghi vào thư mục Magento.

```
# Kiểm tra owner hiện tại
ls -la /var/www/xgear-deploy

# Thêm jenkins vào group www-data
sudo usermod -aG www-data jenkins

# Đổi owner thư mục deploy sang jenkins (KHÔNG phải "deploy" -
# vì user "deploy" chỉ dùng cho kịch bản SSH ở Phần 1, không tồn tại vai trò ở đây)
sudo chown -R jenkins:www-data /var/www/xgear-deploy
sudo chmod -R g+rwX /var/www/xgear-deploy

sudo systemctl restart jenkins
```

Cấp sudo giới hạn cho user `jenkins`

```
sudo visudo -f /etc/sudoers.d/jenkins-deploy
```

```
jenkins ALL=(ALL) NOPASSWD: /usr/sbin/service php8.1-fpm reload
jenkins ALL=(ALL) NOPASSWD: /usr/bin/systemctl reload nginx
jenkins ALL=(ALL) NOPASSWD: /usr/bin/chown -R www-data\:www-data /var/www/xgear-deploy/releases/*
```

(Sửa `php8.1-fpm` đúng version PHP đang dùng.)

Kiểm tra Jenkins gọi được PHP/Composer

```
sudo -u jenkins php -v
sudo -u jenkins composer -V
```

Nếu lỗi `command not found`, thêm PATH cho Jenkins:

```
sudo systemctl edit jenkins
```

```
[Service]
Environment="PATH=/usr/local/bin:/usr/bin:/bin"
```

```
sudo systemctl daemon-reload
sudo systemctl restart jenkins
```

Kiểm chứng quyền ghi thực tế

```
sudo -u jenkins touch /var/www/xgear-deploy/releases/test.txt \
&& echo "OK - ghi được" \
&& sudo -u jenkins rm /var/www/xgear-deploy/releases/test.txt
```

6. Tạo subdomain + SSL cho Jenkins

6.1. Tạo DNS record trên Cloudflare

Cloudflare Dashboard → chọn domain → DNS → Records → Add record:

```
Type:      A
Name:      jenkins
IPv4 address: <VPS_APP_IP>
Proxy status: Proxied (đám mây cam)
TTL:      Auto
```

Kiểm tra DNS đã lan truyền:

```
dig jenkins.thenewxgear.com.vn +short
```

Nếu gặp vướng mắc khi xin SSL do Cloudflare Proxy, tạm chuyển sang **DNS only** (đám mây xám) trong lúc chạy Certbot, bật lại Proxy sau khi xong.

6.2. Tạo file nginx config — CHỈ phần HTTP trước

Thứ tự đúng: tạo config HTTP (port 80) trước, chạy Certbot sau — không làm ngược lại, nếu không Certbot sẽ báo lỗi “Could not automatically find a matching server block”.

```
sudo nano /etc/nginx/sites-available/jenkins
```

```
server {
    listen 80;
    server_name jenkins.thenewxgear.com.vn;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
sudo ln -s /etc/nginx/sites-available/jenkins /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

Kiểm tra trước khi chạy Certbot:

```
curl -I http://jenkins.thenewxgear.com.vn
```

(Nhận được phản hồi bất kỳ, kể cả lỗi 502, là đủ — mục đích chỉ xác nhận nginx đã nhận diện đúng domain.)

6.3. Chạy Certbot xin SSL

```
sudo certbot --nginx -d jenkins.thenewxgear.com.vn
```

Kết quả mong đợi:

```
Successfully received certificate.
Deploying certificate
Successfully deployed certificate for jenkins.thenewxgear.com.vn ...
```

Certbot sẽ tự động sửa file `/etc/nginx/sites-available/jenkins`, thêm các dòng `ssl_certificate`, `ssl_certificate_key`, và tạo thêm server block redirect HTTP → HTTPS.

6.4. Kiểm tra file config sau khi Certbot tự sửa

```
sudo cat /etc/nginx/sites-available/jenkins
```

Kết quả đúng dạng:

```
server {
    server_name jenkins.thenewxgear.com.vn;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    listen 443 ssl; # managed by Certbot
    ssl_certificate /etc/letsencrypt/live/jenkins.thenewxgear.com.vn/fullchain.pem; # managed by Certbot
    ssl_certificate_key /etc/letsencrypt/live/jenkins.thenewxgear.com.vn/privkey.pem; # managed by Certbot
    include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
    ssl_dhparam /etc/letsencrypt/ssl-dhparam.pem; # managed by Certbot
}

server {
    if ($host = jenkins.thenewxgear.com.vn) {
        return 301 https://$host$request_uri;
    } # managed by Certbot

    listen 80;
    server_name jenkins.thenewxgear.com.vn;
    return 404; # managed by Certbot
}
```

```
sudo nginx -t
sudo systemctl reload nginx
curl -I https://jenkins.thenewxgear.com.vn
```

Phải trả về `HTTP/2 200` hoặc `HTTP/2 302`. Mở trình duyệt kiểm tra ổ khóa SSL hợp lệ.

6.5. Xác nhận renew tự động

```
sudo certbot certificates
sudo certbot renew --dry-run
```

7. Hai lựa chọn bảo mật UI Jenkins — chọn 1 trong 2

OPTION A — Không dùng Basic Auth (đơn giản, phù hợp giai đoạn học)

Giữ nguyên file nginx config như Bước 6.4, không sửa gì thêm. Bảo mật dựa vào: - Jenkins vẫn có trang login riêng - Port 8080 chỉ bind `127.0.0.1` (Bước 8), không truy cập trực tiếp được từ ngoài

Ưu điểm: đơn giản, không cần quản lý thêm file password. **Nhược điểm:** endpoint `/github-webhook/` mở public hoàn toàn — ai cũng gửi POST giả kích hoạt build được (dù không đọc được kết quả build vì vẫn cần login Jenkins).

Bỏ qua sang Bước 8 nếu chọn Option A.

OPTION B — Dùng Basic Auth (khuyến nghị cho môi trường thật sự production)

7.1. Cài công cụ tạo file password

```
sudo apt install -y apache2-utils
```

7.2. Tạo user/password bảo vệ

```
sudo htpasswd -c /etc/nginx/.htpasswd youradmin
```

(Đổi `youradmin` theo ý muốn, nhập password khi được hỏi.)

Lưu ý cờ `-c`: chỉ dùng `-c` lần đầu tạo file. Thêm user thứ 2 về sau thì **bỏ `-c`**:

```
sudo htpasswd /etc/nginx/.htpasswd second_user
```

Dùng lại `-c` sẽ ghi đè, xóa mất user cũ.

7.3. Sửa nginx config — thêm Basic Auth, loại trừ path webhook

```
sudo nano /etc/nginx/sites-available/jenkins
```

```
server {
    server_name jenkins.thenewxgear.com.vn;

    # Webhook GitHub - KHÔNG yêu cầu Basic Auth (GitHub không tu nhập được)
    location /github-webhook/ {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # UI Jenkins - YÊU CẦU Basic Auth
    location / {
        auth_basic "Restricted Area";
        auth_basic_user_file /etc/nginx/.htpasswd;

        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_redirect http:// https://;
        proxy_read_timeout 90s;
    }

    listen 443 ssl; # managed by Certbot
    ssl_certificate /etc/letsencrypt/live/jenkins.thenewxgear.com.vn/fullchain.pem; # managed by Certbot
    ssl_certificate_key /etc/letsencrypt/live/jenkins.thenewxgear.com.vn/privkey.pem; # managed by Certbot
    include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
    ssl_dhparam /etc/letsencrypt/ssl-dhparam.pem; # managed by Certbot
}

server {
    if ($host = jenkins.thenewxgear.com.vn) {
        return 301 https://$host$request_uri;
    } # managed by Certbot

    listen 80;
    server_name jenkins.thenewxgear.com.vn;
    return 404; # managed by Certbot
}
```

Về thứ tự location: nginx tự ưu tiên khớp path cụ thể/dài hơn (`/github-webhook/`) so với path gốc (`/`) theo nguyên tắc “prefix match” — không quan trọng khai báo trước hay sau trong file, nhưng nên viết `/github-webhook/` trước cho dễ đọc.

```
sudo nginx -t
sudo systemctl reload nginx
```

7.4. Test Basic Auth

```
# UI Jenkins - phải bị chặn hỏi password
curl -I https://jenkins.thenewxgear.com.vn/

# Webhook path - KHÔNG bị hỏi password
curl -I https://jenkins.thenewxgear.com.vn/github-webhook/
```

Dòng đầu phải trả 401. Dòng hai trả mã khác 401 (thường 403 / 405 vì Jenkins từ chối GET, chỉ nhận POST — đây là bình thường).

8. Chặn truy cập trực tiếp vào port 8080

Dù chọn Option A hay B, **bắt buộc** làm bước này — nếu không, ai cũng có thể bỏ qua toàn bộ nginx bằng cách gõ thẳng `http://<VPS_APP_IP>:8080`.

```
sudo systemctl edit jenkins
```

```
[Service]
Environment="JENKINS_LISTEN_ADDRESS=127.0.0.1"
```

```
sudo systemctl daemon-reload
sudo systemctl restart jenkins
```

Kiểm tra:

```
sudo ss -tlnp | grep 8080
```

Phải hiện `127.0.0.1:8080`, **không được** hiện `0.0.0.0:8080`.

Kiểm tra từ máy ngoài (không phải VPS):

```
curl -I http://<VPS_APP_IP>:8080 --connect-timeout 5
```

Phải bị “Connection refused” hoặc timeout.

Rà soát thêm firewall:

```
sudo ufw status
sudo ufw delete allow 8080 # chỉ chạy nếu trước đó có rule này
sudo ufw allow 443
sudo ufw allow 80
sudo ufw allow 22
```

9. Kết nối Jenkins với Git repo

9.1. Tạo credential Git (nếu repo private)

GitHub → Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token → scope `repo` → copy token.

Jenkins: Manage Jenkins → Credentials → System → Global credentials → Add Credentials: - Kind: Username with password - Username: username GitHub - Password: personal access token - ID: `git-xgear-token`

9.2. Tạo Pipeline Job

1. **New Item** → tên `deploy-xgear-vps` → chọn **Pipeline** → OK
2. **Pipeline** → **Definition:** `Pipeline script from SCM`
 - SCM: `Git`
 - Repository URL: `<GIT_REPO_URL>`
 - Credentials: `git-xgear-token`
 - Branches to build: `*/main`
 - Script Path: `Jenkinsfile`

10. Jenkinsfile hoàn chỉnh (bản chung VPS, không cần SSH)

Đây là bản đã sửa qua thực tế debug — khác 2 điểm quan trọng so với thiết kế ban đầu: 1. `env.php` dùng trực tiếp từ `shared/`

(không qua file dummy `env.php.jenkins`) — xem lý do đầy đủ ở mục 17.8. 2. **Stage dọn dẹp generated/ được đặt TRƯỚC Composer Install**, không phải sau — xem lý do đầy đủ ở mục 18.3 (đây là lỗi rất dễ mắc, phải hiểu đúng cơ chế Composer autoload classmap mới tránh được).

```
pipeline {
    agent any

    environment {
        DEPLOY_BASE      = '/var/www/xgear-deploy'
        RELEASE_NAME     = "${new Date().format('yyyyMMddHHmmss')}"
        RELEASE_PATH     = "${DEPLOY_BASE}/releases/${RELEASE_NAME}"
        KEEP_RELEASES    = 5
        PHP_FPM_SERVICE  = 'php8.2-fpm'
    }

    options {
        timestamps()
        disableConcurrentBuilds()
        buildDiscarder(logRotator(numToKeepStr: '20'))
        timeout(time: 20, unit: 'MINUTES')
    }

    stages {

        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Dọn dẹp generated code cũ') {
            steps {
                sh '''
                    rm -rf generated/code generated/metadata
                    rm -rf var/di var/generation var/cache var/page_cache var/view_preprocessed
                    mkdir -p generated/code generated/metadata var/di var/generation var/cache var/page_cache
                '''
            }
        }

        stage('Composer Install') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'magento-marketplace-keys',
                    usernameVariable: 'MAGENTO_PUBLIC_KEY',
                    passwordVariable: 'MAGENTO_PRIVATE_KEY'
                )]) {
                    sh '''
                        export COMPOSER_AUTH="{\"http-basic\":{\"repo.magento.com\":
                        {\"username\":\"\"$MAGENTO_PUBLIC_KEY\"\", \"password\":\"\"$MAGENTO_PRIVATE_KEY\"\"}}}"
                        composer install --no-dev --optimize-autoloader --prefer-dist
                    '''
                }
            }
        }

        stage('Copy env.php that tu shared') {
            steps {
                sh 'cp /var/www/xgear-deploy/shared/app/etc/env.php app/etc/env.php'
            }
        }

        stage('DI Compile') {
            steps {
                sh 'php bin/magento setup:di:compile'
            }
        }

        stage('Static Content Deploy') {
            steps {
                sh 'php bin/magento setup:static-content:deploy -f vi_VN en_US'
            }
        }

        stage('Copy vào thư mục release mới') {
            steps {
                sh """
                    mkdir -p ${RELEASE_PATH}
                    rsync -a --exclude='.git' ./ ${RELEASE_PATH}/
                """
            }
        }

        stage('Link Shared Resources') {
            steps {
                sh """
                    rm -f ${RELEASE_PATH}/app/etc/env.php
                    ln -sfn ${DEPLOY_BASE}/shared/app/etc/env.php ${RELEASE_PATH}/app/etc/env.php

                    rm -rf ${RELEASE_PATH}/pub/media
                    ln -sfn ${DEPLOY_BASE}/shared/pub/media ${RELEASE_PATH}/pub/media
                """
            }
        }
    }
}
```

```

        rm -rf ${RELEASE_PATH}/var/log
        ln -sfn ${DEPLOY_BASE}/shared/var/log ${RELEASE_PATH}/var/log
    ""
}

stage('Enable Maintenance Mode') {
    steps {
        sh "cd ${RELEASE_PATH} && php bin/magento maintenance:enable"
    }
}

stage('DB Upgrade & Reindex') {
    steps {
        sh ""
        cd ${RELEASE_PATH}
        php bin/magento setup:upgrade --keep-generated
        php bin/magento indexer:reindex
        php bin/magento cache:flush
    ""
    }
}

stage('Switch Symlink (Go Live)') {
    steps {
        sh "ln -sfn ${RELEASE_PATH} ${DEPLOY_BASE}/current"
    }
}

stage('Disable Maintenance & Reload Services') {
    steps {
        sh ""
        cd ${DEPLOY_BASE}/current
        php bin/magento maintenance:disable
        sudo service ${PHP_FPM_SERVICE} reload
        sudo systemctl reload nginx
    ""
    }
}

stage('Fix Permissions') {
    steps {
        sh "sudo chown -R www-data:www-data ${RELEASE_PATH}"
    }
}

stage('Cleanup Old Releases') {
    steps {
        sh ""
        cd ${DEPLOY_BASE}/releases
        ls -lt | tail -n +${KEEP_RELEASES} | xargs -r rm -rf
    ""
    }
}

post {
    failure {
        echo 'Deploy that bai - kiem tra Console Output, can nhac rollback bang job riêng.'
    }
    success {
        echo "Deploy thanh cong: release ${RELEASE_NAME}"
    }
}
}

```

Giải thích: “source folder” nằm ở đâu?

Lệnh `checkout scm` **không nhận tham số đường dẫn** — Jenkins tự động chọn workspace dựa theo tên job:

```
/var/lib/jenkins/workspace/deploy-xgear-vps/
```

Mọi lệnh `sh` phía sau trong cùng pipeline **mặc định chạy trong đúng thư mục này** — không cần `cd` tường minh. Dấu `./` trong lệnh `rsync -a ./ ${RELEASE_PATH}/` chính là workspace đó.

Thư mục này do Jenkins tự quản lý, không phải nơi bạn thao tác Git thủ công (nơi đó là thư mục bạn tự clone riêng, ví dụ `/var/www/xgear-source`, chỉ dùng để soạn `Jenkinsfile`, rồi push lên GitHub).

Vì sao stage “Don dep generated code cu” phải đứng TRƯỚC Composer Install

Đây là điểm rất dễ hiểu sai, đã từng gây lỗi khó debug thực tế (chi tiết đầy đủ ở mục 18.3). Tóm tắt cơ chế:

`composer install --optimize-autoloader` không chỉ cài package — nó **quét toàn bộ filesystem tại thời điểm chạy** để sinh ra `vendor/composer/autoload_classmap.php`, ghi cứng đường dẫn từng class. Nếu lúc Composer chạy mà thư mục `generated/code/` **vẫn còn tồn tại** (tàn dư từ lần build trước), Composer sẽ ghi cứng luôn cả những đường dẫn đó vào classmap. Nếu sau đó mới chạy dọn dẹp (xóa `generated/`), classmap vẫn còn trỏ tới file đã bị xóa — `DI Compile` load theo classmap này sẽ báo lỗi `include(...): Failed to open stream:`

No such file or directory dù generated/ đã được dọn “sạch sẽ”.

Nguyên tắc: luôn dọn generated/ , var/di , var/generation **trước** khi Composer chạy, để Composer không bao giờ quét thấy và ghi cứng đường dẫn tới các file sẽ bị xóa sau đó.

11. Chuẩn bị env.php — dùng file thật từ shared/ , không tạo bản dummy

Đã đổi cách tiếp cận so với thiết kế ban đầu. Ban đầu tài liệu này dùng 1 file env.php.jenkins chứa cấu hình DB giả (dummy) để di:compile chạy được mà không cần lộ dữ liệu thật. Cách này có 2 nhược điểm gặp phải trong thực tế: (1) static-content:deploy cần đọc bảng theme từ DB thật, DB dummy không có dữ liệu này sẽ gây lỗi Access denied ; (2) nếu Jenkins chạy remote (không chung VPS với Magento) thì càng phải tự dựng thêm 1 DB dummy riêng, tốn công vô ích. Giải pháp thực tế hơn: vì Jenkins đang chung VPS với Magento, env.php thật vốn đã có sẵn trên đĩa — chỉ cần dùng thẳng, không bao giờ đưa qua Git.

Cách hoạt động

```
stage('Copy env.php thật từ shared') {
  steps {
    sh 'cp /var/www/xgear-deploy/shared/app/etc/env.php app/etc/env.php'
  }
}
```

File env.php không đi qua Git ở bất kỳ bước nào — nó luôn nằm cố định tại /var/www/xgear-deploy/shared/app/etc/env.php trên server, được copy thẳng vào workspace lúc build, dùng xong thì workspace đó cũng chỉ tồn tại nội bộ trên VPS (không public). di:compile và static-content:deploy chỉ đọc DB, không ghi — dùng thẳng DB production ở bước build là an toàn.

Bảo mật quyền đọc file

```
sudo chmod 640 /var/www/xgear-deploy/shared/app/etc/env.php
sudo chown jenkins:www-data /var/www/xgear-deploy/shared/app/etc/env.php
```

Xác nhận Jenkins đọc được:

```
sudo -u jenkins cat /var/www/xgear-deploy/shared/app/etc/env.php | head -3
```

Khi nào vẫn cần bản dummy / template — nếu Jenkins chạy remote (không chung VPS)

Nếu sau này tách Jenkins ra máy khác (không chung server với Magento), không dùng cách trên được nữa vì Jenkins không truy cập trực tiếp được shared/app/etc/env.php . Khi đó có 2 lựa chọn chuẩn hơn cách dummy:

- Jenkins “Secret file” Credential:** upload nguyên file env.php thật vào Jenkins Credentials (Kind: Secret file), dùng withCredentials([file(...)]) để lấy ra lúc build — không qua Git, không cần dựng DB dummy.
- Template + biến môi trường:** commit 1 file env.php.template chỉ chứa cấu trúc với placeholder (\${DB_HOST} , \${DB_PASS} ...), dùng envsubst kết hợp Jenkins Credentials (mỗi giá trị nhẩy cảm 1 credential riêng) để sinh ra env.php thật lúc build.

Cả 2 cách này phù hợp hơn nhiều so với việc tự dựng DB dummy, vì không cần đồng bộ dữ liệu theo thời gian và không có rủi ro lệch theme/config so với thực tế.

12. Cấu hình GitHub Webhook

12.1. Tạo webhook cơ bản (không dùng secret)

Vào repo GitHub → Settings → Webhooks → Add webhook:

Trường	Giá trị
Payload URL	https://jenkins.thenewxgear.com.vn/github-webhook/
Content type	application/json
SSL verification	Enable SSL verification
Which events	Just the push event
Active	tick

12.2. Tùy chọn nâng cao: thêm Secret cho webhook

Điền secret vào ô **Secret** trong form trên GitHub (một chuỗi ký tự ngẫu nhiên tự đặt, ví dụ tạo bằng `openssl rand -hex 20`).

Lưu ý quan trọng: chỉ điền secret ở phía GitHub là **chưa đủ** — Jenkins phải được cấu hình biết secret đó thì mới thực sự verify chữ ký HMAC (header `X-Hub-Signature-256`). Nếu bỏ qua bước cấu hình phía Jenkins, GitHub vẫn gửi kèm chữ ký nhưng Jenkins **không kiểm tra**, webhook vẫn hoạt động bình thường — chỉ là secret “không có tác dụng thật”.

Để secret có tác dụng thật, cấu hình phía Jenkins:

Bước A — Tạo Credential chứa secret: 1. Manage Jenkins → Credentials → System → Global credentials → Add Credentials 2. Kind: **Secret text** 3. Secret: dán đúng chuỗi đã điền bên GitHub 4. ID: `github-webhook-secret` 5. Create

Bước B — Gắn secret vào cấu hình GitHub plugin:

Vị trí UI có thể khác nhau tùy version Jenkins. Cách tìm chắc chắn nhất: 1. Vào **Manage Jenkins**, dùng ô **Search** ở đầu trang, gõ `GitHub` 2. Hoặc vào **Manage Jenkins → System** (một số version gọi là “Configure System”), dùng `Ctrl+F` tìm `GitHub` 3. Tìm nút **“Advanced”** (icon quyển sổ + bút chì) trong phần cấu hình GitHub Server 4. Mở rộng ra sẽ thấy ô **“Shared secrets”** → Add → chọn credential `github-webhook-secret` 5. Save

Nếu không thấy mục GitHub nào trong System Configuration, kiểm tra lại plugin đã cài đúng tên **“GitHub Plugin”** chưa (Manage Jenkins → Plugins → Installed), cài xong cần restart Jenkins.

Đây là bước tùy chọn — bỏ qua hoàn toàn không ảnh hưởng chức năng chính của webhook, chỉ thiếu lớp xác thực chữ ký chống giả mạo request.

12.3. Bật trigger trong Jenkins job

Job `deploy-xgear-vps` → **Configure** → **Build Triggers** → tick:

GitHub hook trigger for GITScm polling

Save.

13. Test webhook thật

13.1. Push commit thử

```
cd /var/www/xgear-source
echo "// test webhook $(date)" >> app/etc/env.php.jenkins
git add .
git commit -m "test: trigger webhook"
git push origin main
```

13.2. Kiểm tra trên GitHub

Repo → **Settings** → **Webhooks** → webhook vừa tạo → tab **Recent Deliveries**: - ✓ xanh + Response `200` → thành công - ✗ đỏ → xem bảng lỗi bên dưới

Response code	Nguyên nhân	Cách sửa
401	Basic Auth đang chặn nhầm path webhook (nếu chọn Option B)	Kiểm tra lại <code>location /github-webhook/</code> trong nginx config
404	Sai URL, thiếu dấu <code>/</code> cuối	Sửa lại <code>.../github-webhook/</code>
Timeout	DNS/firewall chặn	<code>dig jenkins.thenewxgear.com.vn</code> , <code>curl -I https://jenkins...</code> từ máy ngoài
502 Bad Gateway	nginx không kết nối được Jenkins	<code>systemctl status jenkins</code> , <code>ss -tlnp grep 8080</code>
403	Plugin GitHub chưa cài đúng	Kiểm tra lại Manage Jenkins → Plugins

13.3. Xác nhận Jenkins nhận build tự động

Jenkins → job `deploy-xgear-vps` → **Build History** — build mới xuất hiện trong vài giây sau khi push. Click vào → **Console Output** theo dõi pipeline.

14. Job Rollback (bản chung VPS)

```
pipeline {
  agent any
  parameters {
    string(name: 'RELEASE_TO_ROLLBACK', defaultValue: '', description: 'De trong = quay ve release lien truoc')
  }
  environment {
    DEPLOY_BASE = '/var/www/xgear-deploy'
  }
  stages {
    stage('Rollback') {
      steps {
        sh '''
          cd ${DEPLOY_BASE}/releases
          TARGET="${RELEASE_TO_ROLLBACK}"
          if [ -z "$TARGET" ]; then
            TARGET=$(ls -lt | sed -n 2p)
          fi
          echo "Rolling back to: $TARGET"
          ln -sf ${DEPLOY_BASE}/releases/$TARGET ${DEPLOY_BASE}/current
          sudo service php8.1-fpm reload
        '''
      }
    }
  }
}
```

15. Giới hạn tài nguyên Jenkins (tránh ảnh hưởng site đang chạy)

```
sudo systemctl edit jenkins

[Service]
MemoryMax=1500M
CPUQuota=100%

sudo systemctl daemon-reload
sudo systemctl restart jenkins
```

Theo dõi tài nguyên trong lúc build:

```
htop
```

16. Checklist tổng hợp trước khi vận hành thật

- ☐ Jenkins cài trên VPS, plugin Git/Pipeline/Credentials Binding/GitHub đã cài
- ☐ User `jenkins` join group `www-data`, owner `/var/www/xgear-deploy` là `jenkins:www-data`
- ☐ Sudoers cấp đúng NOPASSWD cho reload php-fpm, nginx, chown releases
- ☐ DNS subdomain `jenkins.<DOMAIN>` đã trỏ đúng, SSL Certbot cấp thành công
- ☐ (Option B) Basic Auth hoạt động cho `/`, không áp dụng cho `/github-webhook/`
- ☐ Port 8080 chỉ bind `127.0.0.1`, không truy cập được trực tiếp từ ngoài
- ☐ `env.php.jenkins` đã có trong repo, `.gitignore` chặn đúng `env.php` thật
- ☐ `Jenkinsfile` (bản không SSH) đã push lên repo, Script Path job trỏ đúng
- ☐ Webhook GitHub tạo xong, “Recent Deliveries” trả `200`
- ☐ (Tùy chọn) Secret webhook đã cấu hình đủ cả 2 phía GitHub và Jenkins nếu muốn verify thật
- ☐ Job Rollback đã tạo và test thử ít nhất 1 lần
- ☐ Jenkins đã giới hạn `MemoryMax` / `CPUQuota`

17. Xử lý lỗi thực tế: Composer 401 khi tải package `repo.magento.com`

Đây là lỗi thực tế đã gặp khi build lần đầu trên VPS, ghi lại chi tiết để tham khảo khi gặp lại.

17.1. Triệu chứng

Log dừng ở stage `Composer Install`, giữa lúc đang tải các package Braintree/PayPal:

```
- Downloading paypal/module-braintree-customer-balance (4.7.0)
```

```
In AuthHelper.php line 225:  
The 'https://repo.magento.com/archives/magento/composer-dependency-version-audit-plugin/magento-composer-dependency-version-audit-plugin-0.1.6.0.zip'  
URL required authentication (HTTP 401).  
You must be using the interactive console to authenticate
```

17.2. Nguyên nhân gốc

Các package `paypal/module-braintree-*` thuộc kho riêng của Adobe/Magento (`repo.magento.com`), không phải Packagist công khai — cần **Magento Marketplace Authentication Keys** (Public key + Private key) để tải.

Trên máy dùng để code hằng ngày, key này thường đã lưu sẵn trong `auth.json` ở home directory. Nhưng Jenkins chạy composer dưới **user hệ thống jenkins**, với home riêng biệt:

```
sudo -u jenkins composer config -g home  
# => /var/lib/jenkins/.config/composer (Composer 2.3+ dùng chuẩn XDG, không còn ~/.composer/)
```

Thư mục này trống, chưa từng có `auth.json` → Composer không biết cách xác thực với `repo.magento.com` → lỗi `401`.

17.3. Cách kiểm tra xác nhận đúng nguyên nhân

```
sudo -u jenkins composer config -g home  
sudo ls -la $(sudo -u jenkins composer config -g home)
```

Nếu không thấy file `auth.json` trong thư mục trả về — đây chính xác là nguyên nhân.

Kiểm tra key còn hợp lệ hay không bằng `curl` trực tiếp (không phụ thuộc Composer):

```
curl -u "<PUBLIC_KEY>:<PRIVATE_KEY>" -I \  
"https://repo.magento.com/archives/magento/composer-dependency-version-audit-plugin/magento-composer-dependency-version-audit-plugin-0.1.6.0.zip"
```

- `HTTP/2 200` hoặc `302` → key đúng, vấn đề nằm ở cách cấu hình chứ không phải key sai
- `HTTP/2 401` → key sai/hết hạn, cần lấy lại từ marketplace.magento.com

17.4. Cách fix — 3 phương án

Phương án 1 (khuyến nghị): Jenkins Credentials + biến môi trường `COMPOSER_AUTH` trong Jenkinsfile

Tạo credential trong Jenkins (Kind: Username with password, Username = Public key, Password = Private key, ID: `magento-marketplace-keys`), sau đó sửa stage `Composer Install`:

```
stage('Composer Install') {  
    steps {  
        withCredentials([usernamePassword(  
            credentialsId: 'magento-marketplace-keys',  
            usernameVariable: 'MAGENTO_PUBLIC_KEY',  
            passwordVariable: 'MAGENTO_PRIVATE_KEY'  
        )) {  
            sh '''  
                export COMPOSER_AUTH="{\"http-basic\":{\"repo.magento.com\":  
{"username\":\"\"$MAGENTO_PUBLIC_KEY\"\", \"password\":\"\"$MAGENTO_PRIVATE_KEY\"\"}}}'  
                composer install --no-dev --optimize-autoloader --prefer-dist  
            '''  
        }  
    }  
}
```

Ưu điểm: key không nằm trần trên đĩa, không lộ trong Console Output (Jenkins tự che giá trị `withCredentials`), dễ thu hồi/đổi khi cần.

Phương án 2: Tạo `auth.json` global 1 lần thủ công trên VPS

```
sudo -u jenkins composer config -g http-basic.repo.magento.com "<PUBLIC_KEY>" "<PRIVATE_KEY>"
```

Không cần sửa Jenkinsfile, nhưng key nằm cố định plaintext trên đĩa — kém an toàn hơn Phương án 1.

Phương án 3: Tạo `auth.json` ngay trong stage compose bằng lệnh `composer config -g` (biến thể của Phương án 1)

```
stage('Cau hình Magento auth.json') {
  steps {
    withCredentials([usernamePassword(
      credentialsId: 'magento-marketplace-keys',
      usernameVariable: 'MAGENTO_PUBLIC_KEY',
      passwordVariable: 'MAGENTO_PRIVATE_KEY'
    )]) {
      sh 'composer config -g http-basic.repo.magento.com "$MAGENTO_PUBLIC_KEY" "$MAGENTO_PRIVATE_KEY"'
    }
  }
}
```

Ghi lại `auth.json` global dùng chung cho mọi build sau, không cần export `COMPOSER_AUTH` mỗi lần.

17.5. Lỗi phát sinh tiếp theo: “Could not find credentials entry”

Sau khi sửa Jenkinsfile theo Phương án 1/3, có thể gặp lỗi mới:

```
ERROR: Could not find credentials entry with ID 'magento-marketplace-keys'
Finished: FAILURE
```

Nguyên nhân: credential được tham chiếu trong Jenkinsfile (`credentialsId: 'magento-marketplace-keys'`) nhưng **chưa thực sự được tạo** trong Jenkins Credentials Store — dễ nhầm lẫn vì việc test `composer install` thành công qua SSH thủ công (tự export biến môi trường) **hoàn toàn độc lập** với việc Jenkins Credentials Store có chứa ID đó hay không. Hai việc này không liên quan nhau về mặt hệ thống.

Cách kiểm tra chắc chắn credential đã tồn tại — dùng Script Console (đáng tin cậy hơn dò UI, vì UI khác nhau tùy version):

Vào `https://<jenkins-domain>/script`, chạy:

```
import com.cloudbees.plugins.credentials.CredentialsProvider
import com.cloudbees.plugins.credentials.common.StandardCredentials
import jenkins.model.Jenkins

def creds = CredentialsProvider.lookupCredentials(
  StandardCredentials.class,
  Jenkins.instance,
  null,
  null
)

creds.each { c -> println "ID: ${c.id} | Class: ${c.class.simpleName}" }
if (creds.isEmpty()) { println "KHÔNG CÓ CREDENTIAL NAO TON TAI TRONG JENKINS" }
```

Nếu không thấy đúng dòng `ID: magento-marketplace-keys` trong kết quả, credential chưa tồn tại — cần tạo lại.

Cách tạo credential chắc chắn — 2 lựa chọn:

Qua UI (URL trực tiếp, tránh bấm nhầm/thoát giữa chừng qua nhiều màn hình):

```
https://<jenkins-domain>/manage/credentials/store/system/domain/_/newCredentials
```

Điền Kind: `Username with password`, Username/Password là Public/Private key, ID: `magento-marketplace-keys`, bấm **Create** và xác nhận trang tự chuyển về danh sách credentials (dấu hiệu lưu thành công).

Qua Script Console (nhANH, không phụ thuộc UI):

```
import com.cloudbees.plugins.credentials.CredentialsScope
import com.cloudbees.plugins.credentials.impl.UsernamePasswordCredentialsImpl
import com.cloudbees.plugins.credentials.CredentialsProvider
import jenkins.model.Jenkins

def store = CredentialsProvider.lookupStores(Jenkins.instance).find {
  it.getDisplayName() == "System"
}

def newCred = new UsernamePasswordCredentialsImpl(
  CredentialsScope.GLOBAL,
  "magento-marketplace-keys",
  "Magento Marketplace Keys",
  "<PUBLIC_KEY>",
  "<PRIVATE_KEY>"
)

store.addCredentials(com.cloudbees.plugins.credentials.domains.Domain.global(), newCred)
println "Đã tạo xong credential: magento-marketplace-keys"
```

17.6. Cách test toàn bộ luồng sau khi fix

- Chạy lại script kiểm tra Script Console, xác nhận đủ credential `magento-marketplace-keys` xuất hiện trong danh sách.
- Vào job → **Build Now** (không bắt buộc phải có commit mới — lỗi này nằm ở tầng cấu hình Jenkins, không liên quan tới code đã push).

- Theo dõi Console Output, xác nhận:
 - Không còn `ERROR: Could not find credentials entry`
 - Stage `Composer Install` không còn `HTTP 401` khi tải `paypal/module-braintree-*`
 - Pipeline tiếp tục chạy qua các stage sau (`DI Compile`, `Static Content Deploy`, ...) thay vì bị `skipped due to earlier failure(s)`.

17.7. Lưu ý bảo mật

Vì Magento Marketplace key từng được dán trực tiếp vào kênh trao đổi (chat/log) trong lúc debug, nên **rotate (đổi) lại key này** trên marketplace.magento.com → My Profile → Access Keys sau khi mọi thứ hoạt động ổn định, rồi cập nhật giá trị mới vào Jenkins Credentials tương ứng.

18. Xử lý lỗi thực tế: `DI Compile` — Redis giả và generated code cũ

Sau khi fix xong lỗi Composer 401, tiếp tục gặp 2 lỗi liên tiếp ở stage `DI Compile` — ghi lại để tham khảo.

18.1. Lỗi 1 — Fake Redis config trong `env.php.jenkins` gây “Connection refused”

Triệu chứng:

```
+ php bin/magento setup:di:compile
Connection to Redis standalone tcp://127.0.0.1:46554 failed after 2 failures. Last Error : (111) Connection refused
```

Nguyên nhân: Để giữ cấu trúc `env.php.jenkins` giống file thật mà không lộ dữ liệu, có thể vô tình khai báo `cache / session / lock` trỏ Redis với **port giả** (ví dụ port ngẫu nhiên không phải `6379` chuẩn). Vì không có Redis service nào thực sự lắng nghe ở port giả đó, `di:compile` cố kết nối và bị từ chối.

Cách fix: không cần giữ cấu trúc Redis “giả” — với môi trường build/compile, đổi hẳn sang **file-based backend**, vẫn đảm bảo không lộ dữ liệu thật:

```
'cache' => [
    'frontend' => [
        'default' => ['backend' => 'Cm_Cache_Backend_File'],
        'page_cache' => ['backend' => 'Cm_Cache_Backend_File']
    ]
]
'session' => [
    'save' => 'files'
]
'lock' => [
    'provider' => 'db'
]
```

`di:compile` không cần cache/session hoạt động thật — chỉ cần không bị lỗi kết nối khi Magento đọc cấu hình. `lock provider` cũng đổi tạm sang `db` (Magento hỗ trợ `db`, `file`, `zookeeper`, không bắt buộc Redis).

Cách test xác nhận: sau khi sửa `env.php.jenkins`, push lên và build lại — log không còn dòng “Connection to Redis” nào.

18.2. Lỗi 2 — `generated/` cũ còn sót lại từ lần build trước

Triệu chứng (xuất hiện sau khi đã fix lỗi Redis):

```
+ php bin/magento setup:di:compile
Warning: include(.../generated/code/Magento/Framework/App/ResourceConnection/Proxy.php):
Failed to open stream: No such file or directory in .../vendor/composer/ClassLoader.php on line 576
```

Nguyên nhân: Jenkins workspace (`/var/lib/jenkins/workspace/deploy-xgear-vps/`) là thư mục **cố định, tái sử dụng qua nhiều lần build**. Thư mục `generated/`, `var/di`, `var/generation` nằm trong `.gitignore`, không được Git quản lý — nếu lần build trước đó thất bại giữa chừng (ví dụ do lỗi Composer hoặc Redis ở các lần build trước), các thư mục này có thể còn ở trạng thái dở dang. Composer’s autoload classmap (sinh ra khi `composer install` báo “Nothing to install, update or remove” vì `vendor/` cũ vẫn còn) trỏ tới các file trong `generated/` chưa từng được tạo đầy đủ, gây lỗi include.

Cách fix: thêm stage dọn dẹp sạch trước khi compile, đặt ngay sau stage chuẩn bị `env.php`:

```
stage('Don dep generated code cu') {
    steps {
        sh '''
            rm -rf generated/code generated/metadata
            rm -rf var/di var/generation var/cache var/page_cache var/view_preprocessed
            mkdir -p generated/code generated/metadata var/di var/generation var/cache var/page_cache
        '''
    }
}
```

Cách test xác nhận: sau khi thêm stage này, build lại — `DI Compile` chạy lâu hơn hẳn lần trước (thường 1-3 phút tùy cấu hình VPS, vì compile lại từ đầu thay vì dừng ngay), kết thúc bằng dòng log dạng `Generated code and dependency injection configuration successfully` thay vì `Warning`.

Ghi chú mở rộng: nếu sau này gặp lỗi tương tự (stale cache) ở stage `Static Content Deploy`, áp dụng cùng nguyên tắc — dọn `pub/static/*` trước khi deploy:

```
sh '''
    rm -rf pub/static/frontend pub/static/adminhtml pub/static/_cache
    php bin/magento setup:static-content:deploy -f vi_VN en_US
'''
```

18.3. Lỗi 3 — Lỗi vẫn tái xuất hiện dù đã có stage dọn dẹp: sai THỨ TỰ stage

Sau khi thêm stage `Don dep generated code cu` (18.2), lỗi `include(...generated/code...): No such file or directory` **vẫn có thể tái xuất hiện** nếu stage dọn dẹp đặt **sau** `Composer Install` thay vì **trước**. Đây là lỗi tinh vi, dễ gây nhầm lẫn vì nhìn qua tưởng đã fix đúng.

Triệu chứng: log cho thấy thứ tự `Composer Install` → `Copy env.php` → `Don dep generated code cu` (đã chạy, không lỗi) → `DI Compile` **vẫn báo lỗi y hệt cũ**:

```
Warning: include(.../generated/code/Magento/Framework/App/ResourceConnection/Proxy.php):
Failed to open stream: No such file or directory
```

Nguyên nhân gốc: `composer install --optimize-autoloader` không chỉ cài package — nó **quét toàn bộ filesystem tại thời điểm nó chạy** để sinh ra `vendor/composer/autoload_classmap.php`, ghi cứng đường dẫn từng class vào file tra cứu tối ưu này. Nếu lúc Composer chạy, thư mục `generated/code/` **vẫn còn tồn tại** (tàn dư từ build trước, chưa bị dọn), Composer sẽ quét thấy và **ghi cứng đường dẫn đó vào classmap**. Ngay sau đó nếu mới chạy `Don dep generated code cu` — xóa đúng file đã bị ghi cứng — thì khi `DI Compile` load theo classmap cũ, file đã không còn tồn tại nữa → lỗi.

Cách fix: đảo thứ tự — đặt stage dọn dẹp `generated/` **trước** `Composer Install`, không phải sau:

```
stage('Checkout') {
    steps { checkout scm }
}

stage('Don dep generated code cu') {
    steps {
        sh '''
            rm -rf generated/code generated/metadata
            rm -rf var/di var/generation var/cache var/page_cache var/view_preprocessed
            mkdir -p generated/code generated/metadata var/di var/generation var/cache var/page_cache
            ...
        '''
    }
}

stage('Composer Install') {
    // ... giữ nguyên nội dung ...
}

stage('Copy env.php that tu shared') {
    // ... giữ nguyên nội dung ...
}

stage('DI Compile') {
    // ... giữ nguyên nội dung ...
}
```

Với thứ tự này, `generated/` luôn trống **trước khi** Composer quét filesystem, nên classmap sinh ra không bao giờ chứa tham chiếu tới file sẽ bị xóa sau đó — loại bỏ hoàn toàn nguồn gốc lỗi, không phải chỉ vá triệu chứng.

Điểm dễ gây hiểu lầm khi debug: vì đây là lỗi phụ thuộc **trạng thái workspace tại thời điểm build**, có thể có những lần build “tình cờ” chạy qua được (nếu build trước đó để lại `generated/` đang trống sẵn), khiến việc debug theo kiểu “thử lại xem có qua không” dễ cho ra kết luận sai. Cách xác minh chắc chắn nhất là kiểm tra đúng **thứ tự stage** trong Jenkinsfile, không dựa vào việc build có pass hay không ở 1-2 lần thử.

Bản Jenkinsfile hoàn chỉnh ở mục 10 đã áp dụng đúng thứ tự này.

19. Xử lý lỗi thực tế: Webhook ngừng tự trigger sau khi Cancel build giữa chừng

19.1. Triệu chứng

Push commit lên GitHub, “Recent Deliveries” của webhook báo **thành công (200)**, nhưng Jenkins **không tạo build mới nào** — không có lỗi, không có log bất thường, chỉ đơn giản là im lặng không có gì xảy ra.

19.2. Nguyên nhân

Xảy ra sau khi bấm **Cancel một build đang chạy giữa chừng** (ví dụ đang ở giữa bước `sh` gọi `composer install` hoặc `php bin/magento ...`). Jenkins ghi kết quả cuối cùng vào `build.xml` là `ABORTED` — về mặt lịch sử/log build đã “đóng” — nhưng tiến trình con (`sh` process) đôi khi **không bị kill ngay lập tức**, khiến executor của Jenkins vẫn giữ trạng thái “đang bận” trong bộ nhớ runtime, không đồng bộ với trạng thái ghi trên đĩa.

Vì Jenkinsfile có `disableConcurrentBuilds()`, executor đang “kẹt” này khiến **mọi build mới (kể cả do webhook trigger) bị xếp vào hàng đợi chờ vô thời hạn**, không bao giờ thực sự chạy — không có exception nào được ném ra, nên rất khó nhận biết qua log thông thường.

19.3. Cách xác nhận

Kiểm tra file `build.xml` của build bị Cancel — dù thấy `<result>ABORTED</result>` (trông có vẻ đã đóng đúng cách), vẫn không loại trừ được khả năng executor bị kẹt, vì 2 trạng thái này (log trên đĩa và executor runtime) có thể lệch nhau:

```
sudo cat /var/lib/jenkins/jobs/~ten-job~/builds/~so-build~/build.xml | grep -E "<result>|<building>"
```

Kiểm tra hàng đợi qua Script Console (`/script`):

```
import jenkins.model.Jenkins
Jenkins.instance.queue.items.each {
    println "Task: ${it.task.name} | Why: ${it.why}"
}
```

19.4. Cách fix

Cách nhanh nhất: vào job, bấm **Build Now** (hoặc **Build with Parameters**) một lần, và **để nó chạy xong hoàn toàn** (dù kết quả fail hay pass) — không Cancel giữa chừng lần này. Việc để build chạy trọn vẹn theo đúng vòng đời sẽ giải phóng đúng cách trạng thái executor đang kẹt. Sau đó, các lần webhook trigger tiếp theo sẽ hoạt động bình thường trở lại.

Nếu cần xử lý ngay không muốn đợi: dùng Script Console để kill cứng:

```
import jenkins.model.Jenkins
def job = Jenkins.instance.getItemByFullName("<ten-job>")
def build = job.getBuildByNumber(<so-build-bị-kẹt>)
if (build != null && build.isBuilding()) {
    build.doStop()
    println "Đã gửi lệnh stop"
}
```

19.5. Cách phòng tránh cho lâu dài — thêm `timeout()`

```
options {
    timestamps()
    disableConcurrentBuilds()
    buildDiscarder(logRotator(numToKeepStr: '20'))
    timeout(time: 20, unit: 'MINUTES')
}
```

Chỉ thị này đảm bảo nếu build bị treo/kẹt bất thường (do Cancel lỡ tay hoặc lỗi hệ thống), Jenkins **tự động abort sau 20 phút** thay vì kẹt vô thời hạn — xử lý đúng gốc rễ vấn đề. Đã áp dụng sẵn trong Jenkinsfile hoàn chỉnh ở mục 10.

Khuyến nghị vận hành: hạn chế bấm Cancel một build đang chạy giữa chừng nếu không thực sự cần thiết — ưu tiên để build tự chạy xong (dù fail). Nếu bắt buộc phải Cancel, nên chủ động bấm Build Now một lần nữa và để nó hoàn tất, thay vì mặc định tin rằng hệ thống đã sạch.

19.6. Vì sao không nên bỏ hẳn `disableConcurrentBuilds()` để né vấn đề này

Dù `disableConcurrentBuilds()` là nguyên nhân gián tiếp gây ra hiện tượng “webhook im lặng” khi có build bị kẹt, **không nên bỏ hẳn chỉ thị này** — pipeline deploy Magento thao tác trực tiếp lên filesystem dùng chung (`releases/`, `current` symlink) và service production, nếu 2 build chạy song song có thể gây hậu quả nghiêm trọng hơn nhiều:

- **Đụng độ `RELEASE_NAME`** : nếu 2 build khởi động cùng 1 giây, cả 2 tạo cùng `RELEASE_PATH`, ghi đè lẫn nhau
- **Workspace dùng chung bị lẫn code**: `checkout scm` của build B có thể ghi đè code đang được build A compile dở
- **Switch Symlink chạy chống chéo**: build B có thể tắt maintenance mode trong khi build A vẫn đang giữa quá trình `DB Upgrade`, gây lỗi 500 cho khách hàng thật và có thể làm sai lệch dữ liệu

Giải pháp đúng là **giữ `disableConcurrentBuilds()`, chỉ thêm `timeout()`** như mục 19.5 — giải quyết đúng vấn đề gốc (build kẹt vô thời hạn) mà không đánh đổi an toàn dữ liệu production.

20. Bảng lỗi thường gặp tổng hợp

Lỗi	Nguyên nhân	Cách sửa
Could not automatically find a matching server block (certbot)	Chạy certbot trước khi tạo file nginx config	Tạo config HTTP (port 80) trước, chạy certbot sau
Permission denied khi Jenkins ghi vào releases/	Owner thư mục sai (là deploy thay vì jenkins)	sudo chown -R jenkins:www-data /var/www/xgear-deploy
sudo: no tty present	Sudoers thiếu NOPASSWD	Kiểm tra lại /etc/sudoers.d/jenkins-deploy
composer: command not found (dưới user jenkins)	PATH của Jenkins thiếu	Thêm Environment="PATH=..." qua systemctl edit jenkins
Webhook trả 401	Basic Auth áp dụng nhầm cho path webhook	Kiểm tra thứ tự/khai báo location /github-webhook/
Webhook trả 502	Jenkins service chưa chạy hoặc sai port proxy	systemctl status jenkins , kiểm tra proxy_pass http://127.0.0.1:8080
Site lỗi 500 sau khi switch symlink	Thiếu quyền đọc hoặc thiếu var/generation	tail -f /var/www/xgear-deploy/current/var/log/exception.log
HTTP 401 khi composer tải repo.magento.com	User jenkins chưa có Magento Marketplace auth.json	Xem chi tiết mục 17 — dùng COMPOSER_AUTH qua Jenkins Credentials
Could not find credentials entry with ID '...'	Credential tham chiếu trong Jenkinsfile chưa thực sự tồn tại trong Jenkins Credentials Store	Xem chi tiết mục 17.5 — xác nhận và tạo lại qua Script Console
Connection to Redis ... Connection refused khi compile	env.php build khai báo cache/session/lock trỏ Redis với port giả	Xem mục 18.1 — đổi sang file-based backend, hoặc dùng env.php thật từ shared/ (mục 11)
Access denied for user 'dummy_user' khi Static Content Deploy	DB dummy không có dữ liệu theme thật	Dùng env.php thật từ shared/ thay vì DB dummy — xem mục 11
Warning: include(.../generated/...): No such file or directory khi compile (vẫn còn dù đã có stage dọn dẹp)	Stage dọn dẹp generated/ đặt SAU Composer Install thay vì TRƯỚC	Xem mục 18.3 — đảo thứ tự stage
Webhook trả 200 nhưng không có build nào chạy	Executor bị kẹt do build trước bị Cancel giữa chừng	Xem mục 19 — Build Now 1 lần cho chạy hết, thêm timeout()
fatal: couldn't find remote ref refs/heads/\${BRANCH}	“Branches to build” dùng */\${BRANCH} nhưng Jenkins chưa từng đọc được Jenkinsfile có tham số này	Xem mục 21.3 — tạm trả về branch cố định, Build Now 1 lần, rồi mới đổi lại
end of line reached within a simple string... (Groovy syntax) khi thêm tham số	Ternary lồng nhau viết trực tiếp trong khối environment {}	Xem mục 21.4 — chuyển logic vào script {} trong 1 stage riêng

21. Mở rộng: thêm tham số chọn Branch + Environment khi Build thủ công

Đây là tính năng mở rộng sau khi pipeline cơ bản đã chạy ổn định — cho phép chọn nhánh Git và môi trường đích (production/uat/develop) ngay khi bấm Build, thay vì cố định main → production như thiết kế ban đầu.

21.1. Cài Git Parameter Plugin

Manage Jenkins → Plugins → Available plugins → tìm “Git Parameter” → cài → restart Jenkins nếu được yêu cầu.

21.2. Jenkinsfile thêm tham số — bản đã sửa lỗi cú pháp, ổn định

```
pipeline {
    agent any

    parameters {
        gitParameter(
            name: 'BRANCH',
            type: 'PT_BRANCH',
            defaultValue: 'main',
            branchFilter: 'origin/(.*)',
            sortMode: 'ASCENDING_SMART',
            description: 'Chọn branch để build và deploy'
        )
        choice(
            name: 'DEPLOY_ENV',
            choices: ['production', 'uat', 'develop'],
            description: 'Chọn môi trường deploy'
        )
    }

    environment {
        PHP_FPM_SERVICE = 'php8.2-fpm'
        KEEP_RELEASES    = 5
    }
}
```

```

options {
    timestamps()
    disableConcurrentBuilds()
    buildDiscarder(logRotator(numToKeepStr: '20'))
    timeout(time: 20, unit: 'MINUTES')
}

stages {

    stage('Checkout') {
        steps {
            checkout scm
        }
    }

    stage('Xac dinh moi truong deploy') {
        steps {
            script {
                if (params.DEPLOY_ENV == 'production') {
                    env.DEPLOY_BASE = '/var/www/xgear-deploy'
                } else if (params.DEPLOY_ENV == 'uat') {
                    env.DEPLOY_BASE = '/var/www/xgear-deploy-uat'
                } else {
                    env.DEPLOY_BASE = '/var/www/xgear-deploy-dev'
                }
                env.RELEASE_NAME = new Date().format('yyyyMMddHHmmss')
                env.RELEASE_PATH = "${env.DEPLOY_BASE}/releases/${env.RELEASE_NAME}"

                echo "Building branch '${params.BRANCH}' -> deploy vao moi truong '${params.DEPLOY_ENV}' (${env.DEPLOY_BASE})"
            }
        }
    }

    stage('Don dep generated code cu') {
        steps {
            sh '''
                rm -rf generated/code generated/metadata
                rm -rf var/di var/generation var/cache var/page_cache var/view_preprocessed
                mkdir -p generated/code generated/metadata var/di var/generation var/cache var/page_cache
            '''
        }
    }

    stage('Composer Install') {
        steps {
            withCredentials([usernamePassword(
                credentialsId: 'magento-marketplace-keys',
                usernameVariable: 'MAGENTO_PUBLIC_KEY',
                passwordVariable: 'MAGENTO_PRIVATE_KEY'
            )]) {
                sh '''
                    export COMPOSER_AUTH="{\"http-basic\":{\"repo.magento.com\":
{"username":"","$MAGENTO_PUBLIC_KEY":"","password":"","$MAGENTO_PRIVATE_KEY"}}}"
                    composer install --no-dev --optimize-autoloader --prefer-dist
                '''
            }
        }
    }

    stage('Copy env.php that tu shared') {
        steps {
            sh 'cp ${DEPLOY_BASE}/shared/app/etc/env.php app/etc/env.php'
        }
    }

    stage('DI Compile') {
        steps {
            sh 'php bin/magento setup:di:compile'
        }
    }

    stage('Static Content Deploy') {
        steps {
            sh 'php bin/magento setup:static-content:deploy -f vi_VN en_US'
        }
    }

    stage('Copy vao thu muc release moi') {
        steps {
            sh '''
                mkdir -p ${RELEASE_PATH}
                rsync -a --exclude='.git' ./ ${RELEASE_PATH}/
            '''
        }
    }

    stage('Link Shared Resources') {
        steps {
            sh '''
                rm -f ${RELEASE_PATH}/app/etc/env.php
                ln -sfn ${DEPLOY_BASE}/shared/app/etc/env.php ${RELEASE_PATH}/app/etc/env.php

                rm -rf ${RELEASE_PATH}/pub/media
                ln -sfn ${DEPLOY_BASE}/shared/pub/media ${RELEASE_PATH}/pub/media
            '''
        }
    }
}

```

```

        rm -rf ${RELEASE_PATH}/var/log
        ln -sfn ${DEPLOY_BASE}/shared/var/log ${RELEASE_PATH}/var/log
        ...
    }
}

stage('Enable Maintenance Mode') {
    steps {
        sh 'cd ${RELEASE_PATH} && php bin/magento maintenance:enable'
    }
}

stage('DB Upgrade & Reindex') {
    steps {
        sh '''
            cd ${RELEASE_PATH}
            php bin/magento setup:upgrade --keep-generated
            php bin/magento indexer:reindex
            php bin/magento cache:flush
        '''
    }
}

stage('Switch Symlink (Go Live)') {
    steps {
        sh 'ln -sfn ${RELEASE_PATH} ${DEPLOY_BASE}/current'
    }
}

stage('Disable Maintenance & Reload Services') {
    steps {
        sh '''
            cd ${DEPLOY_BASE}/current
            php bin/magento maintenance:disable
            sudo service ${PHP_FPM_SERVICE} reload
            sudo systemctl reload nginx
        '''
    }
}

stage('Fix Permissions') {
    steps {
        sh 'sudo chown -R www-data:www-data ${RELEASE_PATH}'
    }
}

stage('Cleanup Old Releases') {
    steps {
        sh '''
            cd ${DEPLOY_BASE}/releases
            ls -lt | tail -n +${((KEEP_RELEASES+1))} | xargs -r rm -rf
        '''
    }
}
}

post {
    failure {
        echo 'Deploy that bai - kiem tra Console Output, can nhac rollback bang job rieng.'
    }
    success {
        echo "Deploy thanh cong: branch=${params.BRANCH}, env=${params.DEPLOY_ENV}, release=${env.RELEASE_NAME}"
    }
}
}

```

Điểm khác biệt so với bản không tham số (mục 10): - Thêm khối `parameters {}` khai báo `BRANCH` (Git Parameter, tự dò danh sách nhánh thật trên remote) và `DEPLOY_ENV` (Choice, chọn 1 trong 3 môi trường) - Logic chọn `DEPLOY_BASE` theo môi trường đặt trong `script {}` của 1 stage riêng (*Xác định môi trường deploy*), **không** viết trực tiếp trong khối `environment {}` — lý do ở mục 21.4 - Toàn bộ `sh "..."` (double-quote, Groovy interpolation) đổi sang `sh '...''` (single-quote) để Bash tự đọc biến môi trường lúc runtime, tránh vướng các vấn đề parse chuỗi phức tạp của Groovy

21.3. Lỗi “con gà quả trứng” khi đổi Branches to build thành biến động

Triệu chứng:

```
fatal: couldn't find remote ref refs/heads/${BRANCH}
```

Nguyên nhân: với job “Pipeline script from SCM”, Jenkins cần 1 lần **checkout khám phá** (đọc nội dung Jenkinsfile để biết stage/tham số) *trước khi* build chính thức bắt đầu — bước này dùng đúng giá trị điền trong ô **Branches to build**. Tại thời điểm đó, tham số `BRANCH` **chưa được gán giá trị nào** (build chưa thực sự bắt đầu), nên nếu “Branches to build” đã là `*/${BRANCH}`, Jenkins giữ nguyên chuỗi literal `${BRANCH}`, không có nhánh nào tên vậy tồn tại → lỗi.

Cách khắc phục — đúng thứ tự: 1. Đảm bảo **Branches to build** đang là `*/main` (cố định) khi push Jenkinsfile có `parameters {}` mới 2. Bấm **Build Now**, để chạy xong (dù pass/fail) — mục đích chỉ để Jenkins đọc và đăng ký thành công cấu trúc tham số vào job 3. Sau đó, nút chuyển thành **Build with Parameters** — xác nhận tham số đã đăng ký 4. Lúc này mới đổi **Branches to build** thành `*/${BRANCH}`, Save 5.

Từ giờ dùng **Build with Parameters** để chọn branch + môi trường mỗi lần build tay

Lưu ý: mỗi lần sửa Jenkinsfile làm thay đổi cấu trúc `parameters {}` (thêm/bớt tham số), cần lặp lại đúng trình tự 1-4 này — nếu không muốn lặp lại thường xuyên, có thể giữ “Branches to build” cố định `main` vĩnh viễn và tự viết `checkout` động ngay trong Jenkinsfile (dùng `params.BRANCH` bên trong 1 stage, không phụ thuộc cấu hình SCM ở Configure job).

21.4. Lỗi cú pháp Groovy khi viết ternary lồng trong `environment {}`

Triệu chứng:

```
org.codehaus.groovy.control.MultipleCompilationErrorsException: startup failed:
WorkflowScript: 21: end of line reached within a simple string 'x' or "x" or /x/;
solution: for multi-line literals, use triple quotes '''x''' or """x""" or /x/ or $/x/$
```

Nguyên nhân: khối `environment {}` trong Declarative Pipeline rất kén cú pháp — không xử lý tốt biểu thức Groovy phức tạp (ternary lồng nhau, xuống dòng giữa chuỗi) viết trực tiếp bên trong `"${...}"`:

```
// SAI - gay loi cu phap
DEPLOY_BASE = "${params.DEPLOY_ENV == 'production' ? '/var/www/xgear-deploy' :
               params.DEPLOY_ENV == 'uat' ? '/var/www/xgear-deploy-uat' :
               '/var/www/xgear-deploy-dev'}"
```

Cách khắc phục: chuyển toàn bộ logic tính toán ra khỏi `environment {}`, đặt vào `script {}` trong 1 stage riêng, gán qua `env.TEN_BIEN = ...` (cú pháp Groovy thường, không bị giới hạn như trong `environment {}`):

```
stage('Xac dinh moi truong deploy') {
    steps {
        script {
            if (params.DEPLOY_ENV == 'production') {
                env.DEPLOY_BASE = '/var/www/xgear-deploy'
            } else if (params.DEPLOY_ENV == 'uat') {
                env.DEPLOY_BASE = '/var/www/xgear-deploy-uat'
            } else {
                env.DEPLOY_BASE = '/var/www/xgear-deploy-dev'
            }
        }
    }
}
```

Bản Jenkinsfile ở mục 21.2 đã áp dụng đúng cách này.

21.5. Chuẩn bị hạ tầng cho môi trường UAT/Dev (nếu dùng `DEPLOY_ENV`)

Mỗi môi trường cần cấu trúc `releases/shared/current` **riêng biệt**, tồn tại sẵn trước khi build chọn môi trường đó:

```
sudo mkdir -p /var/www/xgear-deploy-uat/{releases,shared/app/etc,shared/pub/media,shared/var/log}
sudo mkdir -p /var/www/xgear-deploy-dev/{releases,shared/app/etc,shared/pub/media,shared/var/log}
sudo chown -R jenkins:www-data /var/www/xgear-deploy-uat /var/www/xgear-deploy-dev
```

Mỗi thư mục `shared/app/etc/env.php` cần trỏ đúng DB riêng, domain riêng của môi trường đó — không dùng chung `env.php` với production. Cần thêm nginx vhost riêng cho từng subdomain môi trường (ví dụ `uat.thenewxgear.com.vn`).

21.6. Cân nhắc chặn webhook tự động deploy nhằm môi trường (tùy chọn, không bắt buộc)

Vì webhook GitHub luôn dùng giá trị mặc định (`BRANCH=main`, `DEPLOY_ENV=production` — phần tử đầu trong `choices`), mọi push code (dù lên branch nào) khi trigger tự động **đều sẽ cố deploy vào production**. Nếu muốn chặn việc này, có thể thêm 1 stage kiểm tra nguồn gốc build (`currentBuild.getBuildCauses(...)`) để chỉ cho phép webhook tự động chạy khi đúng branch `main`, còn lại bắt buộc phải build tay có chọn tham số rõ ràng.

Lưu ý quan trọng nếu áp dụng: biến `env.BUILD_CAUSE` **không tồn tại sẵn** trong Jenkins — cách đúng là dùng API `currentBuild.getBuildCauses('<ten-class-day-du>')`. Tên class cause chính xác **phụ thuộc vào version plugin** đang cài, nên cần xác nhận qua Script Console trước khi tin dùng, thay vì giả định cứng 1 tên class.

Đây là tính năng đã thử nghiệm và **revert lại** trong quá trình xây dựng pipeline này do phát sinh thêm việc cần xác minh (tên class Cause đúng), chưa được áp dụng chính thức vào Jenkinsfile ở mục 21.2 — cân nhắc bổ sung sau khi đã vận hành ổn định với tham số cơ bản.

Tài liệu này là Phần 2, tiếp nối Phần 1 (Jenkins chạy trên máy local với SCM Polling). Cả hai dùng chung cấu trúc `releases/shared/current` trên VPS Magento — phần khác biệt chính nằm ở cách Jenkins kết nối tới VPS (qua SSH ở Phần 1, chạy nội bộ ở Phần 2) và cách trigger build (Poll SCM ở Phần 1, Webhook tức thời ở Phần 2).